

Cleartext vs Encrypted

A Packet-Level Comparison Lab

Hemant Sharma

Home lab: Lubuntu VM in VirtualBox, Wireshark, vsftpd, OpenSSH

September 2026

What this lab is about

I ran the same kind of network session twice - once over cleartext protocols (FTP) and once over encrypted ones (SSH, HTTPS) - captured both in Wireshark, and then put them side by side. The goal was fairly simple: see, at the packet level, why the protocol you pick decides whether your credentials end up on the wire for anyone to read.

Experiment 1 - A password flying by in cleartext (FTP)

The setup here was pretty basic. I started Wireshark on the loopback interface and logged in to a vsftpd FTP server running on the same machine. The password I typed was a throwaway one on purpose - the point was to see what the network reveals, not to actually authenticate.



```
logi@logi-virtualbox: ~  
Session Actions Edit View Help  
logi@logi-virtualbox: ~ x logi@logi-virtualbox: ~ x  
logi@logi-virtualbox:~$ ftp localhost  
Trying [::1]:21 ...  
Connected to localhost.  
220 (vsFTPd 3.0.5)  
Name (localhost:logi): logi  
331 Please specify the password.  
Password:  
530 Login incorrect.  
ftp: Login failed  
ftp> bye  
221 Goodbye.  
logi@logi-virtualbox:~$
```

Figure 1: The FTP login on the terminal. The password isn't echoed to the screen, but that only hides it from the console - not from the network.

When I followed the TCP stream of that session in Wireshark, everything the terminal had been hiding showed up:

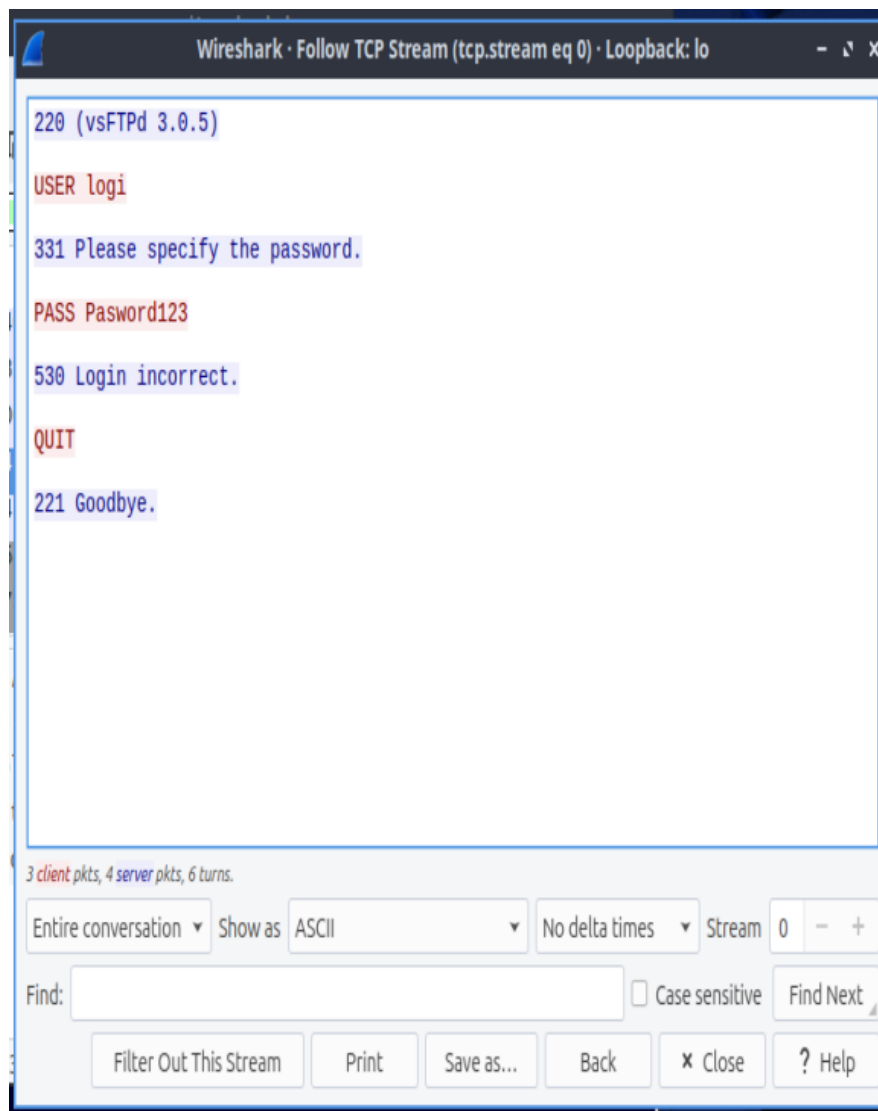
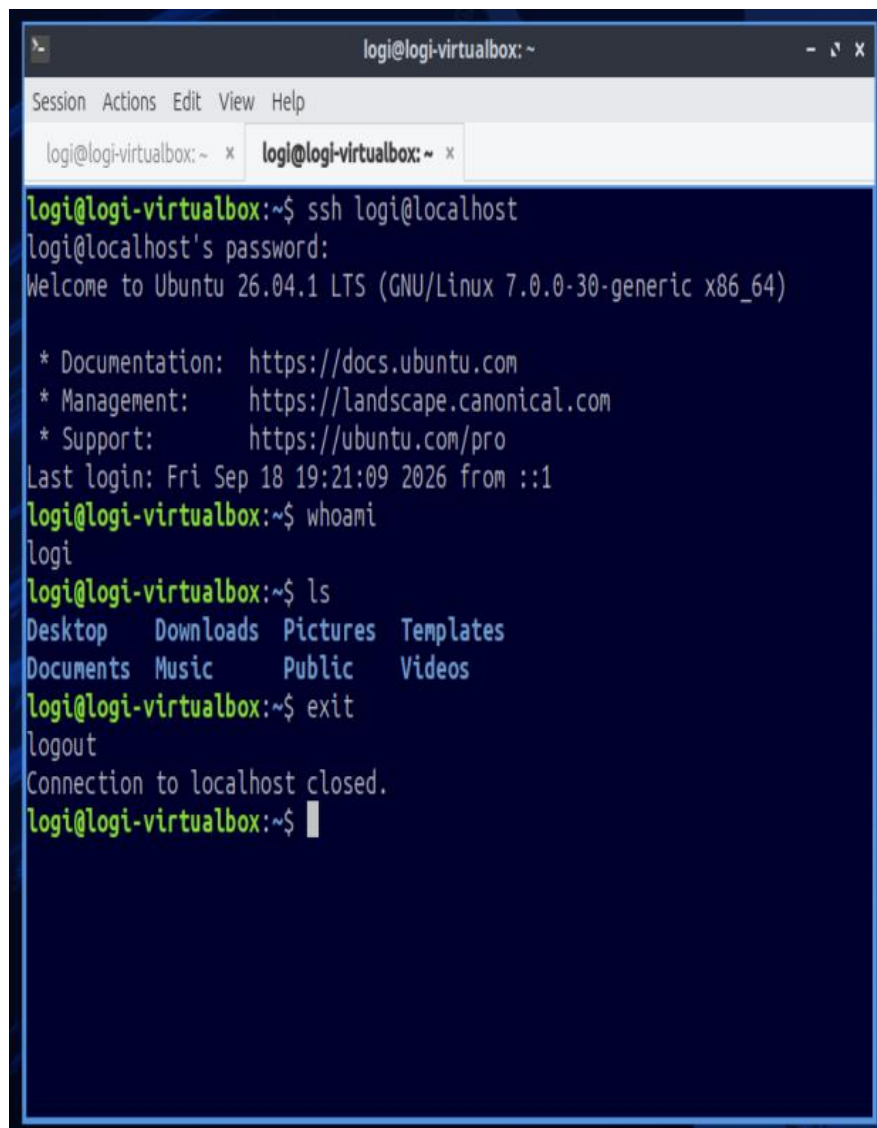


Figure 2: Wireshark - Follow TCP Stream. The whole session reads like plain English: the server banner, the username (USER logi), the password itself (PASS Pasword123), and the server's response.

The part that surprised me most is that even a rejected login gives the password away. The server's verdict doesn't un-send the bytes - the credentials crossed the network before any authentication decision happened. Anyone with a capture on the path has them, whether the login succeeded or not.

Experiment 2 - The same kind of session, encrypted (SSH)

For this one I started a fresh capture on loopback, logged in to the same machine over SSH (ssh logi@localhost), ran a few commands and logged out.



```
logi@logi-virtualbox: ~  
Session Actions Edit View Help  
logi@logi-virtualbox: ~ x logi@logi-virtualbox: ~ x  
logi@logi-virtualbox:~$ ssh logi@localhost  
logi@localhost's password:  
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-30-generic x86_64)  
  
* Documentation:  https://docs.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
Last login: Fri Sep 18 19:21:09 2026 from ::1  
logi@logi-virtualbox:~$ whoami  
logi  
logi@logi-virtualbox:~$ ls  
Desktop  Downloads  Pictures  Templates  
Documents Music    Public    Videos  
logi@logi-virtualbox:~$ exit  
logout  
Connection to localhost closed.  
logi@logi-virtualbox:~$
```

Figure 3: A normal interactive SSH session - login, a few commands, logout.

Following the TCP stream of the SSH session gives about the sharpest contrast with Figure 2 you could ask for:

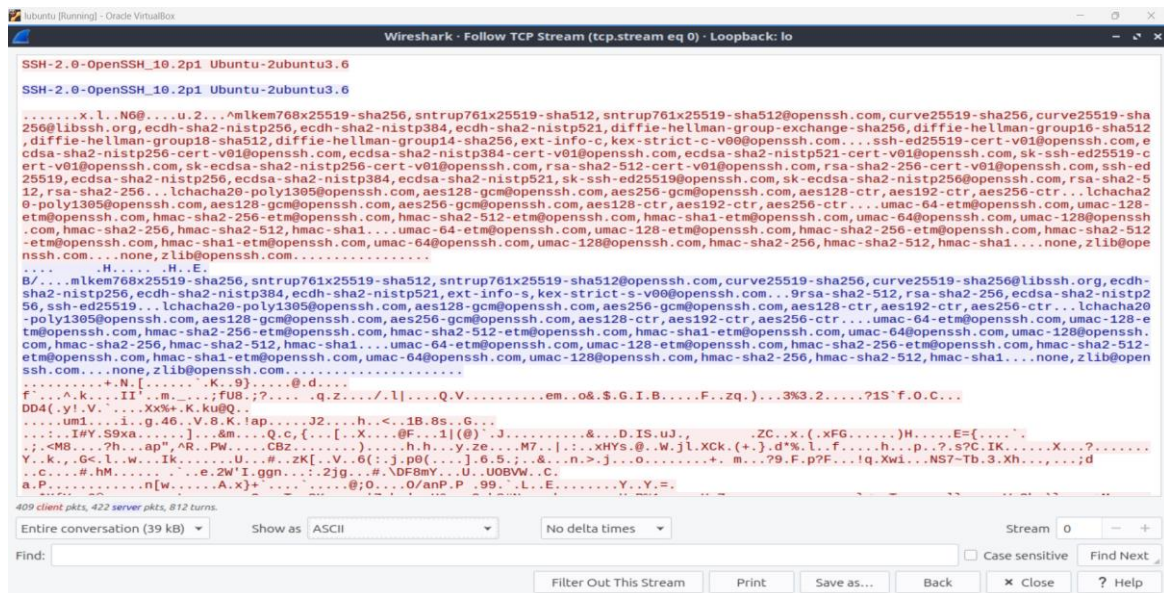


Figure 4: Wireshark - Follow TCP Stream of the SSH session. Only the protocol banner and the cipher negotiation list are readable; everything after that is encrypted binary. That's 39 kB of traffic with no username, password or command anywhere in it.

The readable bit at the start is there by design. Both sides have to openly agree on which encryption algorithms to use before they can encrypt anything - a bit like two people deciding out loud which language to whisper in. Everything sensitive happens after that switch flips. The visible negotiation lists (curve25519-sha256, aes256-gcm and friends) are exactly that agreement happening.

Experiment 3 - Decrypting my own HTTPS session (TLS key logging)

This time the browser was launched with TLS session-key logging turned on (QTWEBENGINE_CHROMIUM_FLAGS="--ssl-key-log-file=\$HOME/ssl-key.log"). I captured traffic to Wikipedia on the main interface, then pointed Wireshark at the key file (Preferences, Protocols, TLS, (Pre)-Master-Secret log filename). Without the keys there's nothing to read:

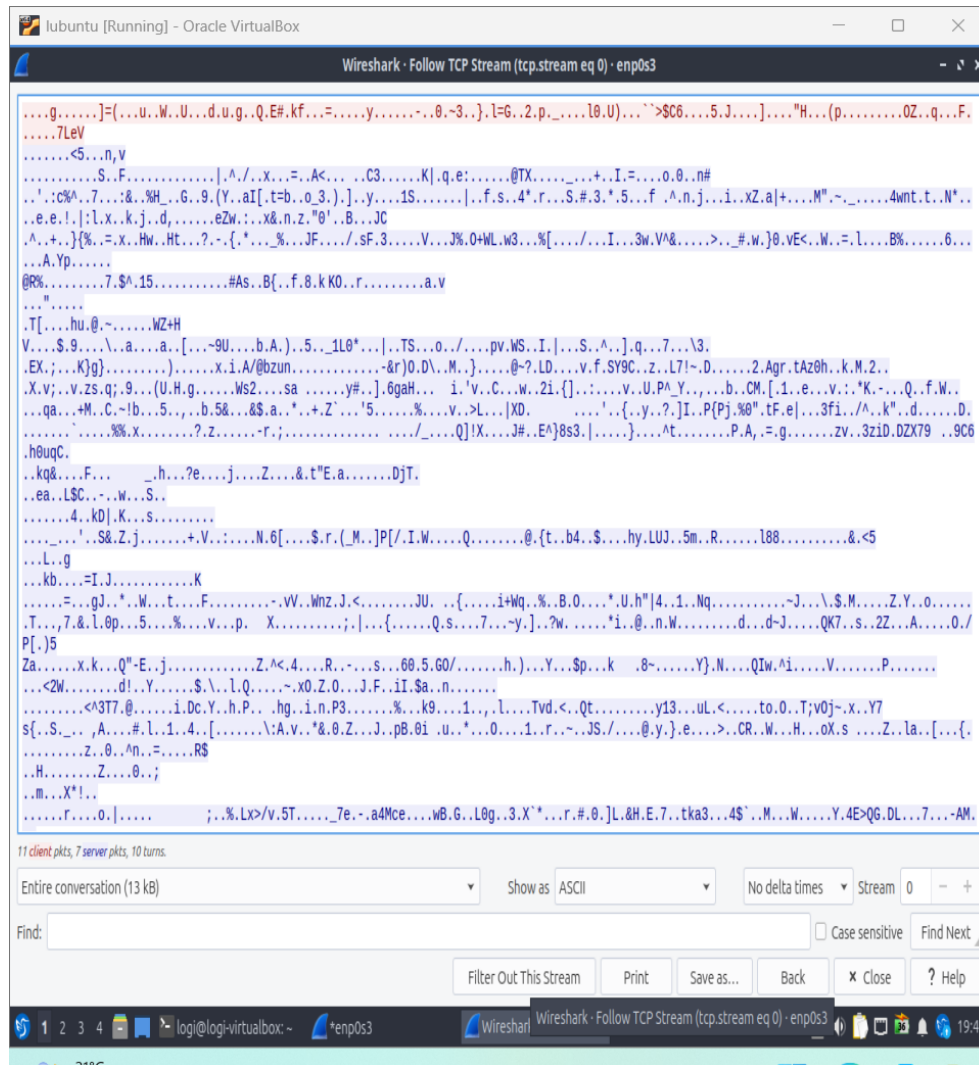


Figure 5: The HTTPS session before decryption - just TLS records, unreadable.

With the session keys loaded, the same capture decrypted into HTTP/2, and the packet dissection showed the request that had been hiding inside the encryption:

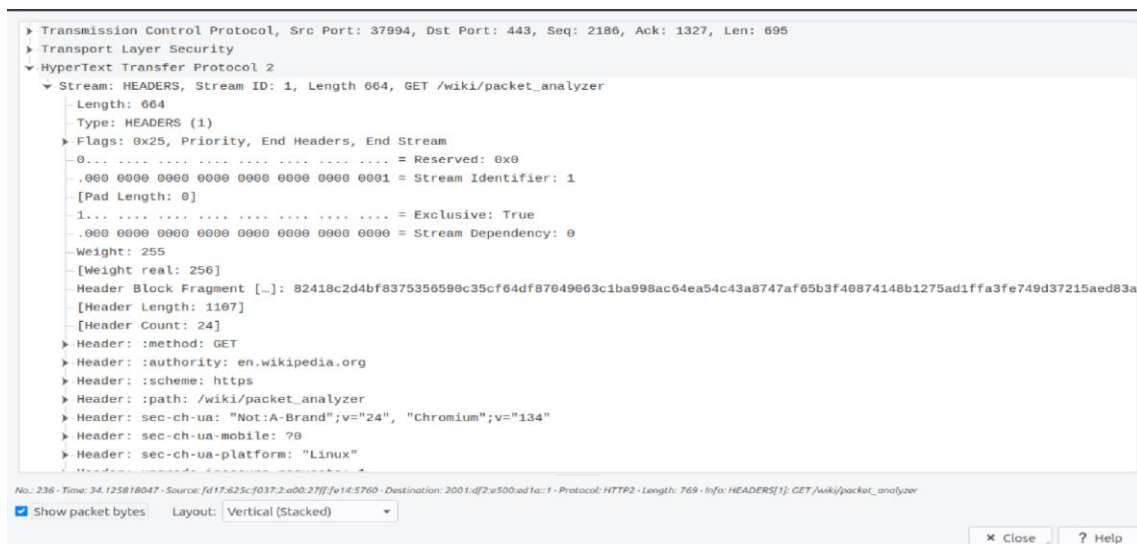


Figure 6: The same session after loading the key file. The dissection pane now shows :method: GET, :authority: en.wikipedia.org and :path: /wiki/Package_analyzer - the browsing is readable again.

What I took from this is that encryption is really key management, not magic. Whoever holds the session keys reads the traffic - which is why this works on my own capture and does nothing against anyone else's. Leaking a key file is as good as leaking the plaintext. (One small quirk of HTTP/2: the decrypted headers are HPACK-compressed, so the readable content shows up in the packet dissection pane rather than the raw stream.)

Lessons

1. Cleartext protocols leak everything, every time - including failed logins.
2. SSH's design is instructive in itself: an open handshake to pick ciphers, and then total encryption of everything sensitive.
3. TLS is only as strong as key custody. With the keys, encrypted becomes readable; without them, it stays noise.
4. Wireshark's Follow TCP Stream is one of the fastest ways to find out what a protocol is actually sending.

Inspired by the networking module of TryHackMe's Cyber Security 101 path (Networking Concepts, Networking Essentials, Networking Core Protocols, Networking Secure Protocols).